



Python Notes



These notes cover digit-based number problems using `while` loops in Python. Each section includes the program name, sample input/output, and the full code.



Key Theory — Digit Extraction

Two core operations power all digit-based problems:

- `n % 10` → Extracts the **last digit** of `n`
- `n // 10` → **Removes** the last digit from `n`

Loop condition: `while n > 0` — stops when all digits are processed.

⚠ **Common Interview Traps:**

- `sum` and `min` are **built-in Python functions** — avoid using them as variable names!
- For **Palindrome**, always save the original number before modifying it.
- Armstrong numbers are also called **Narcissistic numbers**. For 3-digit numbers, cube each digit. For n-digit numbers, raise each digit to the power of n.
- `153`, `370`, `371`, `407` are the most famous 3-digit Armstrong numbers — memorize them!



1. Sum of Digits

Find the sum of all digits of a given number.

Sample Input	Sample Output
1234	10

```
n = int(input())
sum = 0
while(n > 0):
    d = n % 10
    sum += d
    n = n // 10
print(sum)
```

 **Output (Input: 1234)**

10

2. Count Digits

Count the total number of digits in a given number.

Sample Input	Sample Output
12345	5

```
n = int(input())
count = 0
while(n > 0):
    n = n // 10
    count += 1
print(count)
```

 **Output (Input: 12345)**

5



3. Reverse a Number

Reverse the digits of a given number.

Sample Input	Sample Output
1234	4321

```
n = int(input())
rev = 0
while(n > 0):
    d = n % 10
    rev = (rev * 10) + d
    n = n // 10
print(rev)
```

 Output (Input: 1234)

4321



4. Palindrome Number

Check if a number reads the same forwards and backwards.

Sample Input	Sample Output
121	121 Palindrome Number
123	Not a Palindrome Number

```
n = int(input())
org = n
rev = 0
while(n > 0):
    digit = n % 10
    rev = rev * 10 + digit
```

```
n = n // 10
if rev == org:
    print(rev, "Palindrome Number")
else:
    print("Not a Palindrome Number")
```

 **Output (Input: 121)**

121 Palindrome Number

 **Output (Input: 123)**

Not a Palindrome Number

+ 5. Sum of Even Digits

Find the sum of only the even digits in a number.

Sample Input	Sample Output
1234	6

```
n = int(input())
sum = 0
while(n > 0):
    digit = n % 10
    if(digit % 2 == 0):
        sum += digit
    n = n // 10
print(sum)
```

 **Output (Input: 1234)**

6



6. Largest Digit in a Number

Find the largest digit present in a given number.

Sample Input	Sample Output
3947	9

```
n = int(input())
mx = 0
while(n > 0):
    digit = n % 10
    if(digit > mx):
        mx = digit
    n = n // 10
print(mx)
```

 Output (Input: 3947)

9



7. Smallest Digit in a Number

Find the smallest digit present in a given number.

Sample Input	Sample Output
3947	3

```
n = int(input())
min = 9
while(n > 0):
    digit = n % 10
    if(digit < min):
        min = digit
```

```
n = n // 10
print(min)
```

 **Output (Input: 3947)**

3

✖ 8. Product of Digits

Find the product (multiplication) of all digits in a number.

Sample Input	Sample Output
123	6

```
n = int(input())
pro = 1
while(n > 0):
    digit = n % 10
    pro = pro * digit
    n = n // 10
print(pro)
```

 **Output (Input: 123)**

6

🏆 9. Check Armstrong Number

A number is Armstrong if the sum of cubes of its digits equals the number itself.

Example: $153 = 1^3 + 5^3 + 3^3 = 153$ ✓

Sample Input	Sample Output
153	153
123	Not Armstrong Number

```

n = int(input())
org = n
cube = 1
sum = 0
while(n > 0):
    digit = n % 10
    cube = digit * digit * digit
    sum += cube
    n = n // 10
if(sum == org):
    print(sum)
else:
    print("Not Armstrong Number")

```

 **Output (Input: 153)**

153

 **Output (Input: 123)**

Not Armstrong Number

10. Sum of Digits at Even Positions

Find the sum of digits that are at even positions (0-indexed from the right).

Sample Input	Sample Output
12345	9

```
n = int(input())
pos = 0
sum = 0
while(n > 0):
    digit = n % 10
    if(pos % 2 == 0):
        sum += digit
    pos += 1
    n = n // 10
print(sum)
```

 **Output (Input: 12345)**

9



All 10 programs use `while` loops and digit extraction using `% 10` and `// 10`. Mastering this pattern is the key to solving most digit-based problems! 🚀



Pattern Programs — Nested `for` Loops



Key Theory — Pattern Programs

Patterns use nested loops — always remember:

- **Outer loop** → controls the number of **rows**
- **Inner loop** → controls the number of **columns / characters** per row
- `print("*", end=" ")` prints without a newline
- `print()` moves to the next line
- **Space management** is the trickiest part — spaces = `n - i` (for alignment)

⚠ Key Interview Points:

- Time complexity of all pattern programs = **$O(n^2)$** (two nested loops)
- Hollow patterns use a **conditional** inside the inner loop to decide `*` vs space
- Pyramid center column index = `n - 1` (0-indexed)
- For **Diamond** = **Upper Pyramid** + **Inverted Pyramid**
- For **Butterfly** = **Left half mirrored to Right half**



11. Square Pattern

Print a solid square of stars of size $n \times n$.

Sample Input	Sample Output
4	

```
n = int(input())
for i in range(n):          # Outer loop: rows
    for j in range(n):      # Inner loop: columns
```

```

        print("*", end=" ")
    print()                # Move to next line after each row
w

```

 **Output (Input: 4)**

```

* * * *
* * * *
* * * *
* * * *

```

▲ 12. Right Triangle

| Print a right-angled triangle — row i has $i+1$ stars.

Sample Input	Sample Output
4	

```

n = int(input())
for i in range(n):          # Row number
    for j in range(i+1):    # Print i+1 stars in row i
        print("*", end=" ")
    print()

```

 **Output (Input: 4)**

```

*
* *
* * *
* * * *

```

▼ 13. Reverse Triangle

Print a right triangle in reverse — starts with n stars, reduces each row.

Sample Input	Sample Output
4	

```
n = int(input())
for i in range(1, n+1):      # i goes from 1 to n
    for j in range(i, n+1):  # Print (n-i+1) stars
        print("*", end=" ")
    print()
```

 Output (Input: 4)

```
* * * *
* * *
* *
*
```

14. Number Triangle (Column Numbers)

Each row prints column numbers 1, 2, 3... up to the row number.

Sample Input	Sample Output
4	1 1 2 1 2 3 1 2 3 4

```
n = int(input())
for i in range(1, n+1):      # Row number
```

```
for j in range(1, i+1): # Print column numbers 1 to i
    print(j, end=" ")
print()
```

 **Output (Input: 4)**

```
1
1 2
1 2 3
1 2 3 4
```

15. Number Triangle (Row Numbers)

Each row prints its own row number, repeated i times.

Sample Input	Sample Output
4	1 2 2 3 3 3 4 4 4 4

```
n = int(input())
for i in range(1, n+1): # Row number
    for j in range(i): # Print row number i times
        print(i, end=" ")
    print()
```

 **Output (Input: 4)**

```
1
2 2
3 3 3
4 4 4 4
```



16. Pyramid

Centered pyramid — spaces on left decrease, stars increase each row.

Sample Input	Sample Output
4	

```
n = int(input())
for i in range(1, n+1):
    for j in range(n-i):    # Leading spaces = (n - row)
        print(" ", end="")
    for j in range(i):      # Stars = row number
        print("* ", end="")
    print()
```



Output (Input: 4)

```
  *
 * *
* * *
* * * *
```



17. Inverted Pyramid

Inverted pyramid — spaces increase, stars decrease each row.

Sample Input	Sample Output
4	

```

n = int(input())
for i in range(n):
    for j in range(i):      # Leading spaces increase each row
        print(" ", end="")
    for j in range(n-i):    # Stars decrease each row
        print("* ", end="")
    print()

```

 **Output (Input: 4)**

```

* * * *
 * * *
  * *
   *

```

18. Butterfly Pattern

Mirror image pattern — stars grow from outside inward on both halves.

Sample Input	Sample Output
4	

```

n = int(input())
# Upper half
for i in range(n):
    for j in range(i+1):    # Left stars
        print("*", end=" ")
    for k in range(n-(i+1)): # Middle spaces (left)
        print(" ", end=" ")

```

```

        for j in range(n-(i+1)):      # Middle spaces (right)
            print(" ", end=" ")
        for k in range(i+1):          # Right stars
            print("*", end=" ")
        print()
# Lower half
for i in range(n-1):
    for j in range(n-i-1):            # Left stars (decreasing)
        print("*", end=" ")
    for k in range(i+1):              # Middle spaces (left)
        print(" ", end=" ")
    for j in range(i+1):              # Middle spaces (right)
        print(" ", end=" ")
    for k in range(n-(i+1)):          # Right stars (decreasing)
        print("*", end=" ")
    print()

```

 **Output (Input: 4)**

```

*
* *
* * *
* * * *
* * *
* *
*

```

19. Diamond Pattern

Full diamond = Upper pyramid + Inverted pyramid combined.

Sample Input	Sample Output
4	

Sample Input	Sample Output
	<pre> * </pre>

```

n = int(input())
# Upper half (expanding)
for i in range(n):
    for j in range(n-i-1):    # Leading spaces
        print(" ", end=" ")
    for k in range(i+1):      # Left stars
        print("*", end=" ")
    for j in range(i):        # Right stars (mirrors left)
        print("*", end=" ")
    for k in range(n-i-1):    # Trailing spaces
        print(" ", end=" ")
    print()
# Lower half (shrinking)
for i in range(n-1):
    for j in range(i+1):      # Leading spaces
        print(" ", end=" ")
    for k in range(n-i-2):    # Left stars
        print("*", end=" ")
    for j in range(n-i-1):    # Right stars
        print("*", end=" ")
    for k in range(i):        # Trailing spaces
        print(" ", end=" ")
    print()

```

Output (Input: 4)

```

    *
  * * *
* * * * *

```



```

* * * * *
  * * * *
    * * *
      *

```

20. Hollow Square

Only the border of the square is filled with stars; inside is empty.

Sample Input	Sample Output
4	

```

n = int(input())
for i in range(n):
    for j in range(n):
        # Print * on borders (first/last row OR first/last column)
        if(i==0 or i==n-1 or j==0 or j==n-1):
            print("*", end=" ")
        else:
            print(" ", end=" ")    # Interior is empty
    print()

```

 Output (Input: 4)

```

* * * *
*       *
*       *
*       *
* * * *

```

▲ 21. Hollow Pyramid

Only the outline of the pyramid is drawn — interior is hollow.

Sample Input	Sample Output
4	

```
n = int(input())
for i in range(n):
    for j in range(n-i-1):          # Leading spaces
        print(" ", end=" ")
    for k in range(2*i+1):          # Total stars in row = 2i+1
        # Print * only on edges or first/last row; else print space
        if(k==0 or k==2*i or i==0 or i==n-1):
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

 Output (Input: 4)

```
      *
     * *
    *   *
   *     *
  *       *
 *         *
*         *
```



Quick Revision

Digit Problems (while loop):

- Last digit → `n % 10` | Remove last digit → `n // 10`
- Armstrong (3-digit): sum of **cubes** = original → `153, 370, 371, 407`
- Palindrome: `reverse == original`

Pattern Problems (nested for loop):

- Outer loop = **rows**, inner loop = **columns**
- Pyramid spaces = `n - i`, stars = `i`
- Hollow shape: print `*` only when `i==0`, `i==n-1`, `j==0`, or `j==n-1`
- Diamond = Upper pyramid + Inverted pyramid
- All patterns → Time complexity **$O(n^2)$**



Array (List) Problems — Python



Quick Theory

In Python, **Array** mostly means a **List**.

- Access element: `ar[i]`
- Length: `len(ar)`
- Traversal (values): `for x in ar:`
- Traversal (index): `for i in range(len(ar)):`

Standard input format (competitive programming):

- `n` (size)
- Next line: `n` integers

◆ 1. Print First Element

Given a list, print its first element.

Sample Input	Sample Output
4 1 2 3 4	1

```
n = int(input())
ar = list(map(int, input().split()))
print(ar[0])
```

◆ 2. Traversal (Print All Elements)

Print all elements of the list (one per line).

Sample Input	Sample Output
4 1 2 3 4	1 2 3 4

```
n = int(input())
ar = list(map(int, input().split()))

for x in ar:
    print(x)
```

◆ 3. Print Indices of Array

Print indices from 0 to n-1.

Sample Input	Sample Output
4	0
10 21 13 40	1
	2
	3

```
n = int(input())
ar = list(map(int, input().split()))

for i in range(len(ar)):
    print(i)
```

◆ 4. Sum of Array

Find sum of all elements.

Sample Input	Sample Output
5	15
1 2 3 4 5	

```
n = int(input())
ar = list(map(int, input().split()))

s = 0
for x in ar:
    s += x
print(s)
```

◆ 5. Max Element in Array

Print the maximum value present in the list.

Sample Input	Sample Output
5 3 9 2 8 5	9

```
n = int(input())
ar = list(map(int, input().split()))

mx = ar[0]
for x in ar:
    if x > mx:
        mx = x
print(mx)
```

◆ 6. Reverse Array (Print in Reverse Order)

Print elements from last to first.

Sample Input	Sample Output
4 1 2 3 4	4 3 2 1

```
n = int(input())
ar = list(map(int, input().split()))

for i in range(n - 1, -1, -1):
    print(ar[i], end=" ")
```

◆ 7. Sum of Even Elements

Find sum of only even numbers in the array.

Sample Input	Sample Output
6 1 2 3 4 5 6	58

Sample Input	Sample Output
10 21 13 40 8 7	

```

n = int(input())
ar = list(map(int, input().split()))

s = 0
for x in ar:
    if x % 2 == 0:
        s += x
print(s)

```

◆ 8. Count Elements Greater Than X

Count how many elements are strictly greater than x.

Sample Input	Sample Output
5 2 8 19 3 5	2

```

n = int(input())
ar = list(map(int, input().split()))
x = int(input())

count = 0
for v in ar:
    if v > x:
        count += 1
print(count)

```

◆ 9. Second Largest Element (Unique)

Print the second largest unique element.

If it doesn't exist (all same or only 1 unique), print `No second largest`.

Sample Input	Sample Output
6 4 9 9 2 8 8	8
3 7 7 7	No second largest

```
n = int(input())
ar = list(map(int, input().split()))

uniq = list(set(ar))
if len(uniq) < 2:
    print("No second largest")
else:
    uniq.sort()
    print(uniq[-2])
```

◆ 10. Check if Element Exists

If x is present in the list, print `Element Found` else `Not Found`.

Sample Input	Sample Output
5 1 3 5 7 9 7	Element Found

```
n = int(input())
ar = list(map(int, input().split()))
x = int(input())

if x in ar:
    print("Element Found")
```



```
else:
    print("Not Found")
```

◆ 11. First Occurrence Index of X

Print the first index where x occurs, else print -1.

Sample Input	Sample Output
6 5 1 4 1 9 1 1	1

```
n = int(input())
ar = list(map(int, input().split()))
x = int(input())

if x in ar:
    print(ar.index(x))
else:
    print(-1)
```

◆ 12. Count Even and Odd Elements

Count how many even and odd numbers are in the list.

Sample Input	Sample Output
5 1 2 3 4 5	Even count: 2 Odd count: 3

```
n = int(input())
ar = list(map(int, input().split()))

e = 0
o = 0
```

```
for x in ar:
    if x % 2 == 0:
        e += 1
    else:
        o += 1

print("Even count:", e)
print("Odd count:", o)
```

◆ 13. Count Elements Divisible by X

Count how many elements are divisible by x.

Sample Input	Sample Output
6 2 4 6 8 9 10 2	5

```
n = int(input())
ar = list(map(int, input().split()))
x = int(input())

c = 0
for v in ar:
    if v % x == 0:
        c += 1
print(c)
```

◆ 14. Replace Even Elements with 0

Replace every even element with 0 and print the updated list.

Sample Input	Sample Output
5 1 2 3 4 5	[1, 0, 3, 0, 5]

```
n = int(input())
ar = list(map(int, input().split()))

for i in range(len(ar)):
    if ar[i] % 2 == 0:
        ar[i] = 0

print(ar)
```

◆ 15. Find All Indices of X

Print all indices where x occurs (space-separated).

Sample Input	Sample Output
7 1 2 3 2 2 5 2 2	1 3 4 6

```
n = int(input())
ar = list(map(int, input().split()))
x = int(input())

for i in range(n):
    if ar[i] == x:
        print(i, end=" ")
```

◆ 16. Check if All Elements are Positive

If any element is negative, print `Not Positive` , otherwise print `Positive` .

Sample Input	Sample Output
4 1 5 2 9	Positive
4 1 -5 2 9	Not Positive

```

n = int(input())
ar = list(map(int, input().split()))

for x in ar:
    if x < 0:
        print("Not Positive")
        break
    else:
        print("Positive")

```

◆ 17. Difference Between Max and Min

Print (max - min).

Sample Input	Sample Output
5 7 2 9 15	8

```

n = int(input())
ar = list(map(int, input().split()))

mx = ar[0]
mn = ar[0]

for x in ar:
    if x > mx:
        mx = x
    if x < mn:

```

```
mn = x

print(mx - mn)
```

◆ 18. Count Elements in Range [L, R]

Count how many elements lie between L and R (inclusive).

Sample Input	Sample Output
6 1 5 10 7 3 9 4 9	4

```
n = int(input())
ar = list(map(int, input().split()))
L, R = map(int, input().split())

c = 0
for x in ar:
    if L <= x <= R:
        c += 1
print(c)
```

◆ 19. Missing Number in Array (1 to n)

Given n and a list of size n-1 containing numbers from 1..n (one missing), print the missing number.



Idea: Use sum formula $n(n + 1)/2$ and subtract the sum of array.

Sample Input	Sample Output
5 1 2 3 5	4

```
n = int(input())
ar = list(map(int, input().split()))

if len(ar) == n - 1:
    total = n * (n + 1) // 2
    print(total - sum(ar))
else:
    print("Invalid")
```

 **Output (Input shown above)**

4

◆ 20. First Non-Repeating Element in Array

Print the first element that occurs exactly once. If none, print -1.



Using `count()` inside a loop is **$O(n^2)$** . It is okay for small inputs, but for large inputs prefer a frequency dictionary.

Sample Input	Sample Output
7 2 3 2 4 3 5 4	5

```
n = int(input())
ar = list(map(int, input().split()))

for x in ar:
    if ar.count(x) == 1:
```

```
        print(x)
        break
    else:
        print(-1)
```

 **Output (Input shown above)**

5

◆ 21. Move All Zeroes to End

Move all 0s to the end while keeping the order of non-zero elements.



Idea: First collect non-zero elements, then append required zeros.

Sample Input	Sample Output
6 0 1 0 3 12 0	1 3 12 0 0 0

```
n = int(input())
ar = list(map(int, input().split()))

res = []
for x in ar:
    if x != 0:
        res.append(x)

while len(res) < n:
    res.append(0)

print(*res)
```

 **Output (Input shown above)**

1 3 12 0 0 0

◆ 22. Leaders in Array

A leader is an element that is $\geq$ all elements to its right. Print all leaders.



Idea: Traverse from right, maintain `mx`.

Sample Input	Sample Output
6 16 17 4 3 5 2	17 5 2

```
n = int(input())
ar = list(map(int, input().split()))

mx = ar[-1]
print(mx, end=" ")

for i in range(n - 2, -1, -1):
    if ar[i] >= mx:
        mx = ar[i]
        print(ar[i], end=" ")
```

 **Output (Input shown above)**

2 5 17

◆ 23. Leaders in Array (Print in Original Order)

Same as leaders, but print from left to right (original order).

Sample Input	Sample Output
6 16 17 4 3 5 2	17 5 2

```

n = int(input())
ar = list(map(int, input().split()))

mx = ar[-1]
leaders = [mx]

for i in range(n - 2, -1, -1):
    if ar[i] >= mx:
        mx = ar[i]
        leaders.append(ar[i])

print(*leaders[::-1])

```

 **Output (Input shown above)**

17 5 2

◆ 24. Peak Element

Print all elements that are greater than its immediate left and right neighbors.

For edge elements, compare only the available neighbor.

Sample Input	Sample Output
6 1 3 20 4 1 0	20

```

n = int(input())
ar = list(map(int, input().split()))

for i in range(n):

```

```
if (i == 0 or ar[i] > ar[i - 1]) and (i == n - 1 or ar[i] > ar[i + 1]):  
    print(ar[i])
```

 **Output (Input shown above)**

20

◆ 25. Duplicate Element (First Duplicate)

Print the first element which appears more than once.



This approach uses `count()` so it is $O(n^2)$.

Sample Input	Sample Output
6 1 3 4 2 2 5	2

```
n = int(input())  
ar = list(map(int, input().split()))  
  
for x in ar:  
    if ar.count(x) > 1:  
        print(x)  
        break
```

 **Output (Input shown above)**

2

◆ 26. Sort 0s, 1s, 2s (Using sort)

Given an array containing only 0, 1, 2, sort it.

Sample Input	Sample Output
6 2 0 2 1 1 0	0 0 1 1 2 2

```
n = int(input())
ar = list(map(int, input().split()))

ar.sort()
print(*ar)
```

 **Output (Input shown above)**

```
0 0 1 1 2 2
```

◆ 27. Sort 0s, 1s, 2s (Without sort)

Count 0, 1, 2 and rebuild the array.

Sample Input	Sample Output
6 2 0 2 1 1 0	0 0 1 1 2 2

```
n = int(input())
ar = list(map(int, input().split()))

c0 = ar.count(0)
c1 = ar.count(1)
c2 = ar.count(2)

print(*([0] * c0 + [1] * c1 + [2] * c2))
```

 **Output (Input shown above)**

0 0 1 1 2 2

◆ 28. Sum of Digits of All Elements

For each element, sum its digits. Print the total sum.



This is a mix of **array traversal** + **digit extraction** (`% 10` , `// 10`).

Sample Input	Sample Output
3 12 34 5	15

```
n = int(input())
ar = list(map(int, input().split()))

total = 0
for x in ar:
    while x > 0:
        digit = x % 10
        total += digit
        x //= 10

print(total)
```

Output (Input shown above)

15

◆ 29. Maximum Difference (arr[j] - arr[i]) where j > i

Find the maximum value of arr[j] - arr[i] for any j > i.



The given code is **$O(n^2)$** using two loops. For big inputs, an optimized **$O(n)$** method exists using minimum so far.

Sample Input	Sample Output
6 7 1 5 3 6 4	5

```
n = int(input())
ar = list(map(int, input().split()))

mx = float('-inf')
for i in range(n):
    for j in range(i + 1, n):
        if ar[j] - ar[i] > mx:
            mx = ar[j] - ar[i]

print(mx)
```

 **Output (Input shown above)**

5



Quick Revision — Array Problems

- **Missing number (1..n):** `n*(n+1)//2 - sum(arr)`
- **First unique:** frequency (`count()` or dict)
- **Move zeroes:** keep non-zero order, then append zeros
- **Leaders:** traverse from right, maintain max-so-far
- **Peak:** compare with neighbors (handle edges carefully)
- **0/1/2 sort:** count 0,1,2 and rebuild
- **Max difference:** brute force is $O(n^2)$; optimized uses min-so-far



String Problems — Python



Quick Theory

- Traversal: `for ch in s:`
- Frequency: `s.count(ch)` (easy but $O(n)$ each time)
- Useful tricks:
 - Reverse string: `s[::-1]`
 - Sort letters: `sorted(s)`
 - Split words: `s.split()`



1. First Non-Repeating Character

Print the first character that occurs exactly once. If none, print -1.

Sample Input	Sample Output
aabbcdde	c

```
s = input()

for ch in s:
    if s.count(ch) == 1:
        print(ch)
        break
else:
    print(-1)
```

 **Output (Input shown above)**

c

2. Anagram Check

Check if two strings are anagrams (same characters with same frequency).

Sample Input	Sample Output
listen silent	a

```
s1 = input().strip()
s2 = input().strip()

if sorted(s1) == sorted(s2):
    print("a")
else:
    print("n")
```

 **Output (Input shown above)**

a

3. Palindrome String

Print **p** if string is palindrome, else **n**.

Sample Input	Sample Output
madam	p

```
s = input()

if s == s[::-1]:
    print("p")
else:
    print("n")
```

 **Output (Input shown above)**

p

4. Reverse Each Word

Reverse each word in a sentence (keep word order).

Sample Input	Sample Output
hello world	olleh dlrow

```
s = input()
words = s.split()

for w in words:
    print(w[::-1], end=" ")
```


Output (Input shown above)

```
olleh dlrow
```

5. Count Vowels and Consonants

Count only alphabet letters; print vowels and consonants counts.

Sample Input	Sample Output
Notion AI	vowels: 4 consonants: 4

```
s = input()

v = 0
c = 0
for ch in s:
    if ch.isalpha():
        if ch in 'aeiouAEIOU':
            v += 1
        else:
            c += 1

print("vowels: ", v)
print("consonants: ", c)
```

Output (Input shown above)

```
vowels:  4
consonants:  4
```



Quick Revision — String Problems

- **First unique char:** first `ch` where `count(ch) == 1` (simple but can be slow)
- **Anagram:** `sorted(s1) == sorted(s2)`
- **Palindrome:** `s == s[::-1]`
- **Reverse words:** `split()` then reverse each word
- **Vowels/consonants:** check `isalpha()` then vowel set



Array Problem Starter Template

When you see an array question, first decide:

- **What to track?** (sum, count, max/min, indices)
- **What condition?** (even/odd, $> x$, in range)
- **One pass or two pass?** (single loop vs multiple loops)

Common patterns to memorize:

- Running sum / running count
- Max/Min update
- Reverse traversal / two pointers
- Frequency counting (using dict)